Claude Code Architecture Guide

A Comprehensive Framework for Custom LLM Development

> **DRAFT STATUS**: This is a living document under active development. Sections marked with `[PLACEHOLDER]` are incomplete. Last substantive edit: December 23, 2025.

Version: 0.5.0-draft
Author: Daneyon (with Claude)
Last Updated: December 23, 2025

Table of Contents

1. Executive Summary
2. Architecture Philosophy
3. Prerequisites & Git Foundation
4. Schema 1: Global User Configuration
5. Schema 2: Distributable Plugin Project
6. Component Deep Dives
    - 6.1 Memory System (CLAUDE.md) *(Updated v0.5.0 - 5-tier hierarchy)*
    - 6.2 Agent Skills *(Updated v0.4.0)*
    - 6.3 Subagents *(Updated v0.5.0 - built-in agents)*
    - 6.4 Custom Commands
    - 6.5 Hooks
    - 6.6 MCP Integration *(Updated v0.5.0 - new scopes)*
    - 6.7 Knowledge Base Structure
7. Distribution & Marketplace
8. Operational Patterns
    - 8.1 Git Worktree Parallel Execution
    - 8.2 Session Management
    - 8.3 Multi-Agent Collaboration
9. Implementation Workflow
10. Future Enhancements
    - 10.1 SDK & Programmatic Integration
    - 10.2 Enterprise Deployment
    - 10.3 Advanced MCP Development
    - 10.4 CI/CD Integration
11. Appendix A: Glossary
12. Appendix B: References
13. Appendix C: Knowledge Base Deep Dive [PLACEHOLDER]
14. Appendix D: AI Agents Deep Dive [PLACEHOLDER]

---

1. Executive Summary

Purpose

This guide establishes a standardized architecture for building custom LLM solutions using Claude Code. It addresses the separation of concerns between **personal configuration** (global settings that travel with you) and **distributable projects** (shareable plugins with domain-specific capabilities).

Core Insight

Claude Code is not merely a coding assistant—it is a configurable AI platform with filesystem access, tool integration, and extensible capabilities. This architecture leverages Claude Code for any custom LLM use case: technical coding projects, research assistants, domain-specific knowledge bases, or educational tutors.

Two-Schema Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          ARCHITECTURE OVERVIEW                           │
└──────────────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────────────┐
│  SCHEMA 1: GLOBAL USER CONFIGURATION                                     │
│  Location: ~/.claude/                                                    │
│  Purpose: Personal baseline, cross-project preferences                   │
│  Scope: Private to you, applies to ALL projects                          │
├──────────────────────────────────────────────────────────────────────────┤
│  ~/.claude/                                                              │
│  ├── CLAUDE.md              # Your personal system instructions          │
│  ├── settings.json          # User settings                              │
│  ├── rules/                 # Personal modular rules                     │
│  ├── skills/                # Personal skills (available everywhere)     │
│  │   └── your-skill/                                                     │
│  │       └── SKILL.md                                                    │
│  └── agents/                # Personal agents                            │
│      └── your-agent.md                                                   │
└──────────────────────────────────────────────────────────────────────────┘
                                     │
                                     │ inherits / supplements
                                     ▼
┌──────────────────────────────────────────────────────────────────────────┐
│  SCHEMA 2: DISTRIBUTABLE PLUGIN PROJECT                                  │
│  Location: ./your-project/                                               │
│  Purpose: Domain-specific custom LLM, shareable via marketplace          │
│  Scope: Team/community distribution                                      │
├──────────────────────────────────────────────────────────────────────────┤
│  your-plugin/                                                            │
│  ├── .claude-plugin/                                                     │
│  │   └── plugin.json        # Required: marketplace metadata             │
│  ├── CLAUDE.md              # Project system instructions                │
│  ├── CLAUDE.local.md        # Personal project overrides (gitignored)    │
│  ├── .claude/rules/         # Modular project rules                      │
│  ├── .mcp.json              # MCP server configurations                  │
│  ├── commands/              # Custom slash commands                      │
│  ├── agents/                # Project-specific subagents                 │
```

```
│  ├── skills/                # Project-specific skills                    │
│  ├── hooks/                 # Event handlers                             │
│  ├── knowledge/             # Domain knowledge base                      │
│  └── docs/                  # Documentation                              │
└──────────────────────────────────────────────────────────────────────────┘
```

Key Design Principles

Principle	Description
Separation of Concerns	Global config stays personal; project config is distributable
Progressive Disclosure	Load only what's needed; reference additional files via `@imports`
Convention over Configuration	Follow official directory structures; minimize custom conventions
Git-Native Workflow	Version control everything; enable team collaboration
Token Efficiency	Keep CLAUDE.md concise; use skills/agents for complex instructions
Security by Default	Private repos, credential exclusion, pre-publication review

2. Architecture Philosophy

The Memory Hierarchy (5 Tiers)

> **Source**: Manage Claude's memory

Claude Code implements a 5-tier memory hierarchy with clear precedence:

Tier	Location	Purpose	Shared With
1. Enterprise Policy	System paths*	Organization-wide standards	All org users
2. Project Memory	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Team-shared instructions	Team via git
3. Project Rules	`./.claude/rules/*.md`	Modular topic-specific rules	Team via git
4. User Memory	`~/.claude/CLAUDE.md`	Personal preferences (all projects)	Just you
5. Project Local	`./CLAUDE.local.md`	Personal project-specific	Just you

*Enterprise paths:

- macOS: `/Library/Application Support/ClaudeCode/CLAUDE.md`
- Linux: `/etc/claude-code/CLAUDE.md`
- Windows: `C:\Program Files\ClaudeCode\CLAUDE.md`

Precedence Rule: Files higher in the hierarchy load first and take precedence.

Note: `CLAUDE.local.md` files are automatically added to `.gitignore`.

Project Rules (.claude/rules/)

For larger projects, organize instructions into modular files:

```
.claude/rules/
├── code-style.md     # Code formatting
├── testing.md        # Test conventions
└── security.md       # Security requirements
```

All `.md` files in `.claude/rules/` are automatically loaded as project memory.

Path-specific rules using YAML frontmatter:

```
---
paths: src/api/**/*.ts
---

# API Development Rules
- All API endpoints must include input validation
- Use the standard error response format
```

Rules without a `paths` field apply to all files. User-level rules can be placed in `~/.claude/rules/`.

Invocation Patterns

Understanding how Claude Code components are triggered is critical:

Component	Invocation Type	Trigger
Memory (CLAUDE.md)	Automatic	Always loaded at session start
Project Rules	Automatic	Loaded at session start
Skills	Model-invoked	Claude autonomously decides based on task context
Subagents	Model or user-invoked	Auto-delegated or explicitly called
Commands	User-invoked	Explicitly typed (e.g., `/analyze`)
Hooks	Event-driven	Triggered by system events (PreToolUse, PostToolUse, etc.)

Distribution Philosophy

Problem: How do you share a complete working system while keeping global and project layers separate?

Solution: CLAUDE.md `@imports` enable optional personalization without bloating the plugin:

```
# Project CLAUDE.md

## Core Instructions
[Self-contained project instructions]

## Personal Customization (Optional)
@~/.claude/project-preferences.md
```

- If the user creates `~/.claude/project-preferences.md`, their preferences are loaded
- If the file doesn't exist, Claude proceeds without it
- The plugin remains self-contained and functional out of the box

---

3. Prerequisites & Git Foundation

Why Git is Foundational

Git is **architecturally required** for this framework:

Capability	Git Dependency
Plugin distribution	Marketplaces pull from git repositories
Team collaboration	Project CLAUDE.md and configs shared via git
Version control	System instructions, skills, agents all versioned
Parallel execution	Git worktrees enable multiple Claude Code sessions

Initial Setup Checklist

```
# 1. Initialize repository FIRST (before any Claude Code operations)
git init

# 2. Create initial structure
mkdir -p .claude-plugin .claude/rules .claude/skills commands agents knowledge
docs
touch CLAUDE.md README.md .gitignore

# 3. Make initial commit
git add .
git commit -m "Initial project structure"
```

Recommended .gitignore

```
# Claude Code local files
CLAUDE.local.md
.claude.local.json

# Credentials
.env
.env.local
*.key
*.pem
credentials.json

# OS
.DS_Store
Thumbs.db

# IDE
.vscode/
.idea/

# Dependencies (if applicable)
node_modules/
__pycache__/
*.pyc

# Build outputs
dist/
build/
```

GitHub Repository Setup

For distribution, connect to GitHub:

```
# Create repo on GitHub first, then:
git remote add origin https://github.com/yourusername/my-custom-llm.git
git branch -M main
git push -u origin main
```

Privacy & Security Defaults

Critical: Always create repositories as **private by default** during development.

```
# Using GitHub CLI (recommended for security)
gh repo create my-custom-llm --private --source=. --push

# Only make public after explicit review
gh repo edit my-custom-llm --visibility public
```

Pre-publication checklist:

- ☐ No API keys, tokens, or credentials in repository
- ☐ No personal/client data in knowledge base
- ☐ `.gitignore` excludes sensitive files (`.env`, `*.key`, etc.)
- ☐ CLAUDE.md contains no proprietary instructions you don't want shared
- ☐ Review all files for PII or confidential information

Files that should NEVER be committed:

```
.env                     # Environment variables
*.key, *.pem            # Cryptographic keys
settings.local.json     # Local settings overrides
CLAUDE.local.md         # Personal local instructions
credentials.json        # Any credential files
```

GitHub MCP Integration

Connect Claude Code to GitHub for enhanced repository operations:

```
# Add GitHub MCP server
claude mcp add --transport http github https://api.githubcopilot.com/mcp/

# Authenticate via /mcp command in Claude Code
```

---

4. Schema 1: Global User Configuration

Directory Structure

```
~/.claude/
├── CLAUDE.md                    # Personal baseline (always loaded)
├── settings.json                # User settings (model, permissions)
│
├── rules/                       # Personal modular rules (NEW)
│   ├── preferences.md
│   └── workflows.md
│
├── skills/                      # Personal skills (cross-project)
│   ├── research-methodology/
│   │   └── SKILL.md
│   └── technical-writing/
│       └── SKILL.md
│
├── agents/                      # Personal subagents
│   └── general-researcher.md
│
```

```
└── shared-knowledge/             # Reference materials (optional)
    ├── frameworks/
    │   └── llm_architecture_framework.md
    └── templates/
        └── analysis-template.md
```

CLAUDE.md (Personal Baseline)

```
# Personal Configuration

## Communication Style
- Employ systems thinking and comprehensive analysis
- Provide structured overviews before detailed analysis (tables, diagrams, mind
maps)
- Balance theoretical depth with practical applicability
- Use natural, conversational tone unless technical analysis is explicitly needed
- Minimize unnecessary formatting; use prose over bullet points for explanations

## Default Behaviors
- Ask clarifying questions before diving into complex or ambiguous tasks
- Use markdown artifacts for reference-ready materials
- Provide evidence-based reasoning with citations where applicable
- Challenge assumptions constructively; maintain intellectual honesty

## Response Structure Preferences
- Executive summaries for complex topics
- Clear section headings for longer responses
- Actionable next steps where appropriate

## Personal Context
[Your background, expertise areas, common workflows—loaded for context]
```

settings.json

```
{
  "model": "sonnet",
  "permissions": {
    "allow": [
      "Read",
      "Write",
      "Edit",
      "Bash(git *)",
      "Bash(npm *)",
      "Bash(python *)"
    ],
    "deny": []
  }
}
```

5. Schema 2: Distributable Plugin Project

Directory Structure

```
my-custom-llm/
├── .claude-plugin/
│   └── plugin.json              # Required: marketplace metadata
│
├── CLAUDE.md                    # Project system instructions
├── CLAUDE.local.md              # Personal overrides (gitignored)
│
├── .claude/
│   ├── rules/                   # Modular project rules
│   │   ├── code-style.md
│   │   └── testing.md
│   └── skills/                  # Project skills
│       └── domain-skill/
│           └── SKILL.md
│
├── .mcp.json                    # MCP server configurations
│
├── commands/                    # Custom slash commands
│   ├── analyze.md
│   └── report.md
│
├── agents/                      # Project-specific subagents
│   └── domain-expert.md
│
├── hooks/
│   └── hooks.json               # Event handlers
│
├── knowledge/                   # Domain knowledge base
│   ├── INDEX.md
│   ├── core/
│   ├── reference/
│   └── templates/
│
├── docs/                        # Documentation
│   └── README.md
│
└── README.md                    # Project overview
```

plugin.json

```
{
  "name": "my-custom-llm",
  "version": "1.0.0",
  "description": "Domain-specific custom LLM for [purpose]",
  "author": {
```

```
    "name": "Your Name",
    "email": "you@example.com"
  },
  "repository": "https://github.com/yourusername/my-custom-llm",
  "keywords": ["domain", "custom-llm", "claude-code"],
  "license": "MIT"
}
```

CLAUDE.md (Project)

```
# [Project Name]

## Role
You are a [domain] specialist...

## Core Knowledge
@knowledge/INDEX.md

## Key Workflows
1. [Primary workflow]
2. [Secondary workflow]

## Constraints
- [Domain-specific constraints]

## Personal Customization (Optional)
@~/.claude/project-preferences.md
```

---

6. Component Deep Dives

6.1 Memory System (CLAUDE.md)

> **Source**: Manage Claude's memory

CLAUDE.md files serve as persistent memory—instructions that Claude reads at the start of every session.

Memory Hierarchy (5 Tiers)

Tier	Location	Purpose	Shared With
1. Enterprise Policy	System paths	Organization-wide	All org users
2. Project Memory	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Team-shared	Team via git
3. Project Rules	`./.claude/rules/*.md`	Modular rules	Team via git

Tier	Location	Purpose	Shared With
4. User Memory	`~/.claude/CLAUDE.md`	Personal	Just you
5. Project Local	`./CLAUDE.local.md`	Personal project-specific	Just you

Import Syntax

```
See @README.md for project overview.
For architecture details, see @docs/architecture.md

## Personal Preferences (Optional)
@~/.claude/project-preferences.md
```

Rules:

- Both relative and absolute paths are supported
- Imports can chain recursively (max depth: 5 hops)
- Use `/memory` command to see all loaded files
- Imports inside code blocks are ignored

Project Rules (.claude/rules/)

Modular rules for larger projects:

```
.claude/rules/
├── code-style.md
├── testing.md
└── security.md
```

Path-specific rules:

```
---
paths: src/api/**/*.ts
---

# API Development Rules
- All endpoints must include input validation
```

Rules without `paths:` apply to all files.

Best Practices

Do	Don't
Be specific: "Use 2-space indentation"	Be vague: "Format code properly"
Use `.claude/rules/` for modular organization	Put everything in one CLAUDE.md
Keep files under 500 lines	Bloat with rarely-used instructions
Use `@imports` for detailed material	Inline everything

6.2 Agent Skills

> **Source**: Agent Skills

Skills are **model-invoked** capabilities—Claude autonomously decides when to use them based on task context and skill descriptions.

Core Philosophy

> The context window is a public good. Only add context Claude doesn't already have.

Skill Structure

```
skill-name/
├── SKILL.md              # Required: instructions
└── Bundled Resources     # Optional
    ├── scripts/          # Executable code
    ├── references/       # Documentation (loaded on demand)
    └── assets/           # Templates, images, fonts
```

SKILL.md Format

```
---
name: skill-name
description: What this skill does AND when to use it. Include trigger words.
Third-person voice.
allowed-tools: Read, Grep, Glob    # Optional: restrict tools
---

# Skill Name

## Instructions
1. Step-by-step guidance
2. Clear procedures

## Examples
Concrete examples of usage
```

```
## References
For details, see [reference.md](reference.md)
```

Frontmatter Fields

Field	Required	Description
`name`	Yes	Lowercase letters, numbers, hyphens (max 64 chars)
`description`	Yes	What skill does + when to use (max 1024 chars)
`allowed-tools`	No	Restrict which tools skill can use

Critical: The `description` field determines when Claude uses the skill. Include both what it does AND when to use it.

Progressive Disclosure

Level	When Loaded	Token Cost
Metadata	Session start	~100 tokens/skill
Instructions	When triggered	< 5k tokens
Resources	As needed	Unlimited

Skill Locations

Type	Location	Scope
Personal	`~/.claude/skills/`	All projects
Project	`.claude/skills/`	Current project
Plugin	Bundled with plugin	When installed

6.3 Subagents

> **Source**: Subagents

Overview

Subagents are specialized AI assistants that operate in their **own context window**, preventing pollution of the main conversation. Claude can delegate tasks to subagents automatically or users can invoke them explicitly.

File Format

Location: `agents/` directory (project or personal)

```
---
name: my-agent-name
description: Description of when this subagent should be invoked
tools: Read, Grep, Glob, WebSearch    # Optional: comma-separated list
model: sonnet                          # Optional: sonnet, opus, haiku, or
'inherit'
---

# Agent Name

You are a specialized agent focused on [specific purpose].

## Approach
1. [Step 1]
2. [Step 2]
3. [Step 3]

## Constraints
- [Constraint 1]
- [Constraint 2]

## Output Format
[Expected output structure]
```

Configuration Fields

Field	Required	Description
`name`	Yes	Unique identifier (lowercase letters and hyphens)
`description`	Yes	When to invoke this agent (natural language)
`tools`	No	Comma-separated tool list; inherits all if omitted
`model`	No	Model alias or 'inherit'; defaults to configured subagent model
`permissionMode`	No	Permission handling: `default`, `acceptEdits`, `bypassPermissions`, `plan`, `ignore`
`skills`	No	Skills to auto-load when agent starts

Built-in Subagents

> **Source**: Subagents

Claude Code includes built-in subagents:

Agent	Model	Purpose	Tools
General-purpose	Sonnet	Complex multi-step tasks requiring exploration and action	All tools

Agent	Model	Purpose	Tools
Plan	Sonnet	Codebase research during plan mode	Read, Glob, Grep, Bash
Explore	Haiku	Fast read-only codebase searches	Glob, Grep, Read, limited Bash

Explore subagent is optimized for speed with thoroughness levels:

- **Quick** — Fast targeted lookups
- **Medium** — Balanced speed and thoroughness
- **Very thorough** — Comprehensive multi-location analysis

Skills vs Agents Comparison

Aspect	Skills	Subagents
Invocation	Model-invoked (automatic)	Model or user-invoked
Context	Loads into main context	Separate context window
Purpose	Procedural knowledge, capabilities	Specialized assistant, delegation
State	Stateless	Can be resumed with agentId
Token Impact	Adds to main context	Preserves main context

Advanced Features

Resumable Agents: Subagents can be resumed to continue previous work:

```
> Resume agent abc123 and continue the analysis
```

Chaining Agents:

```
> Use code-analyzer to find issues, then use optimizer to fix them
```

CLI-based Configuration (for session-specific agents):

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer.",
    "prompt": "You are a senior code reviewer...",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  }
}'
```

6.4 Custom Commands

Overview

Commands are **user-invoked** shortcuts triggered by typing `/command-name`. They can execute instructions, reference files, run bash commands, or combine multiple actions.

File Format

Location: `commands/` directory

```
---
description: Brief description shown in /help
allowed-tools: Read, Write, Bash    # Optional: restrict tools
---

# Command Instructions

Clear instructions for what Claude should do when this command is invoked.

## Steps
1. [Step 1]
2. [Step 2]

## Arguments
This command accepts: $ARGUMENTS

## Output
[Expected output format]
```

Examples

Analysis Command (`commands/analyze.md`):

```
---
description: Analyze codebase structure and quality
allowed-tools: Read, Grep, Glob, Bash
---

# Code Analysis

Analyze the codebase for:
1. Directory structure and organization
2. Code quality patterns
3. Potential improvements

Arguments: $ARGUMENTS (optional: specific directory or file pattern)
```

```
Output a structured report with findings and recommendations.
```

6.5 Hooks

Overview

Hooks are **event-driven** scripts that execute automatically in response to Claude Code events. They enable validation, formatting, logging, and automation workflows.

Configuration

Location: `hooks/hooks.json`

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write|Edit",
        "hooks": [
          {
            "type": "command",
            "command": "./scripts/format.sh $FILE_PATH"
          }
        ]
      }
    ],
    "UserPromptSubmit": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "./scripts/log-prompt.sh"
          }
        ]
      }
    ]
  }
}
```

Available Events

Event	Trigger	Common Uses
`PreToolUse`	Before any tool	Block dangerous ops, add warnings
`PostToolUse`	After any tool	Format code, validate output

Event	Trigger	Common Uses
`UserPromptSubmit`	User sends message	Log, preprocess, validate
`SessionStart`	Session begins	Environment setup, checks
`SessionEnd`	Session ends	Cleanup, reporting
`PreCompact`	Before context compaction	Save important state

6.6 MCP Integration

> **Source**: Connect Claude Code to tools via MCP

Overview

Model Context Protocol (MCP) connects Claude Code to external tools, databases, and APIs.

> **Quality Measure**: The quality of an MCP server is measured by how well it enables LLMs to accomplish real-world tasks, not just API coverage.

Transport Types

Type	Best For	Notes
HTTP	Remote servers, cloud services	Recommended for production
stdio	Local integrations, CLI tools	Don't log to stdout (use stderr)
SSE	Legacy	Deprecated

Installation

```
# HTTP server (recommended for remote)
claude mcp add --transport http <name> <url>

# Example: Notion
claude mcp add --transport http notion https://mcp.notion.com/mcp

# stdio server (local)
claude mcp add --transport stdio <name> -- <command> [args...]

# Example: Airtable
claude mcp add --transport stdio airtable --env AIRTABLE_API_KEY=YOUR_KEY \
  -- npx -y airtable-mcp-server
```

Note: The `--` separates Claude CLI flags from the server command.

Windows Users: Use `cmd /c` wrapper for npx:

```
claude mcp add --transport stdio my-server -- cmd /c npx -y @some/package
```

MCP Scopes

Scope	Location	Visibility	Flag
Local	`~/.claude.json` (project path)	Just you, current project	`--scope local` (default)
Project	`.mcp.json`	Team via git	`--scope project`
User	`~/.claude.json`	All your projects	`--scope user`

.mcp.json Configuration

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    },
    "local-db": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@company/db-mcp-server"],
      "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

Environment Variable Expansion

```
{
  "mcpServers": {
    "api-server": {
      "type": "http",
      "url": "${API_BASE_URL:-https://api.example.com}/mcp",
      "headers": {
        "Authorization": "Bearer ${API_KEY}"
      }
    }
  }
}
```

Supported syntax:

- `${VAR}` — Expand variable
- `${VAR:-default}` — Use default if not set

Server Naming Conventions

Language	Format	Example
Python	`{service}_mcp`	`slack_mcp`, `github_mcp`
TypeScript	`{service}-mcp-server`	`slack-mcp-server`

Tool Naming

```
# Format: {service}_{action}_{resource}
slack_send_message      # Not: send_message
github_create_issue     # Not: create_issue
```

Why prefix? Prevents tool name conflicts across multiple MCP servers.

Managing Servers

```
# List all servers
claude mcp list

# Get details
claude mcp get github

# Remove server
claude mcp remove github

# Check status (in Claude Code)
/mcp
```

6.7 Knowledge Base Structure

Overview

Knowledge bases provide domain-specific information that Claude can reference. Unlike CLAUDE.md (always loaded) or skills (loaded when triggered), knowledge files are read on-demand.

Recommended Structure

```
knowledge/
├── INDEX.md                        # Entry point with links
│
```

```
├── core/                          # Essential domain knowledge
│   ├── fundamentals.md
│   └── terminology.md
│
├── reference/                     # Supporting materials
│   ├── standards.md
│   └── regulations.md
│
├── examples/                      # Calibration materials
│   ├── good-examples.md
│   └── edge-cases.md
│
└── templates/                     # Reusable templates
    ├── report-template.md
    └── analysis-template.md
```

Access Patterns

Pattern	When to Use	Implementation
Direct Reference	Always-needed content	`@import` in CLAUDE.md
Skill Reference	Procedural knowledge	Link in SKILL.md
On-Demand	Large documents	Claude reads via filesystem

INDEX.md Example

```md
# Knowledge Base Index

## Core Documentation
- [Fundamentals](core/fundamentals.md) — Essential concepts
- [Terminology](core/terminology.md) — Domain glossary

## Reference Materials
- [Standards](reference/standards.md) — Industry standards
- [Regulations](reference/regulations.md) — Compliance requirements

## Examples
- [Good Examples](examples/good-examples.md) — Quality calibration
- [Edge Cases](examples/edge-cases.md) — Exception handling

## Templates
- [Report Template](templates/report-template.md)
- [Analysis Template](templates/analysis-template.md)
```

---

7. Distribution & Marketplace

Plugin Distribution Flow

```
Development → Local Testing → Security Review → Publication
(Private Repo)  (Local market)  (Run checklist)   (GitHub/Marketplace)
```

Local Testing

```
# Add local development marketplace
claude /plugin marketplace add ./dev-marketplace

# Install and test
claude /plugin install my-plugin@dev-marketplace

# Iterate and refine
```

Security Review Checklist

Before making public:

- ☐ No credentials or API keys
- ☐ No personal/client data
- ☐ No proprietary instructions
- ☐ All files reviewed for PII
- ☐ License file included
- ☐ README documentation complete

GitHub Publication

```
# Make repository public
gh repo edit my-custom-llm --visibility public

# Add to community marketplace (if applicable)
# Users install via:
claude /plugin marketplace add https://github.com/yourusername/my-custom-llm
claude /plugin install my-custom-llm@yourusername
```

---

8. Operational Patterns

8.1 Git Worktree Parallel Execution

Run multiple Claude Code sessions simultaneously using git worktrees:

```
# Create worktree for feature work
git worktree add ../project-feature feature/auth
```

```
# Run Claude Code in each worktree
cd ../project-feature && claude

# Main project continues independently
cd ../project && claude

# Clean up when done
git worktree remove ../project-feature
```

Benefits:

- Independent context windows
- Parallel task execution
- Same repository, different branches
- No configuration conflicts

8.2 Session Management

```
# Resume most recent session
claude --continue

# Resume specific session
claude --resume abc123

# List recent sessions
claude --history
```

8.3 Multi-Agent Collaboration

Design agents that work together:

```
# Primary Agent (agents/coordinator.md)
Delegates to specialized agents based on task type.

# Specialist Agents
- agents/researcher.md — Information gathering
- agents/implementer.md — Code implementation
- agents/reviewer.md — Quality assurance
```

---

9. Implementation Workflow

Phase 1: Foundation (Day 1)

```
# Set up global configuration
mkdir -p ~/.claude/{skills,agents,rules,shared-knowledge}
```

```
touch ~/.claude/CLAUDE.md
touch ~/.claude/settings.json

# Choose migration target
# - Existing project with domain knowledge
# - New project from scratch
```

Phase 2: Plugin Creation (Days 2-3)

```
# Initialize plugin structure
git init my-custom-llm && cd my-custom-llm
mkdir -p .claude-plugin .claude/{rules,skills} commands agents knowledge docs

# Create essential files
touch .claude-plugin/plugin.json CLAUDE.md README.md .gitignore

# Initial commit
git add . && git commit -m "Initial plugin structure"
```

Phase 3: Enhancement (Days 4-7)

1. **Add domain knowledge** to `knowledge/`
2. **Create skills** for procedural knowledge
3. **Create subagents** for specialized tasks
4. **Add commands** for common workflows
5. **Configure hooks** for automation
6. **Set up MCP** for external integrations

Phase 4: Distribution (Day 8+)

1. **Local testing** with development marketplace
2. **Security review** using checklist
3. **Documentation** polish
4. **Publication** to GitHub

---

10. Future Enhancements [PLACEHOLDERS]

10.1 SDK & Programmatic Integration [PLACEHOLDER]

> **Status**: To be developed.

Planned content:

- TypeScript SDK usage patterns
- Python SDK integration
- Programmatic session management
- Tool execution outside terminal

10.2 Enterprise Deployment [PLACEHOLDER]

> **Status**: To be developed.

Planned content:

- Enterprise policy configuration
- Team deployment strategies
- Compliance considerations
- Centralized management

10.3 Advanced MCP Development [PLACEHOLDER]

> **Status**: To be developed.

Planned content:

- Custom MCP server development
- Database integration patterns
- API wrapper patterns
- Authentication flows

10.4 CI/CD Integration [PLACEHOLDER]

> **Status**: To be developed.

Planned content:

- GitHub Actions workflows
- GitLab CI/CD integration
- Automated testing with Claude Code
- Deployment pipelines

Appendix A: Glossary

Term	Definition
CLAUDE.md	Memory file containing persistent system instructions
Project Rules	Modular instructions in `.claude/rules/`
Skill	Model-invoked capability packaged as SKILL.md
Subagent	Specialized assistant with separate context window
Command	User-invoked shortcut (`/command-name`)
Hook	Event-driven script for automation
MCP	Model Context Protocol for external tool integration
Plugin	Distributable package containing components

Term	Definition
Marketplace	Registry for discovering and installing plugins
Progressive Disclosure	Loading content only when needed
Token Efficiency	Minimizing context window usage
Model-invoked	Triggered automatically by Claude based on context
User-invoked	Explicitly triggered by user action

Appendix B: References

> **Note**: Water Resources Engineering case studies have been moved to a separate document: `case_studies_water_resources.md`

Documentation Domain Change (2025)

Claude Code documentation moved to a new domain:

- **OLD**: `docs.anthropic.com/en/docs/claude-code/*`
- **NEW**: `code.claude.com/docs/en/*`

Official Claude Code Documentation

Document	URL
Overview	https://code.claude.com/docs/en/overview
Quickstart	https://code.claude.com/docs/en/quickstart
Memory	https://code.claude.com/docs/en/memory
Skills	https://code.claude.com/docs/en/skills
Subagents	https://code.claude.com/docs/en/sub-agents
Plugins	https://code.claude.com/docs/en/plugins
Discover Plugins	https://code.claude.com/docs/en/discover-plugins
Hooks	https://code.claude.com/docs/en/hooks-guide
MCP	https://code.claude.com/docs/en/mcp
Settings	https://code.claude.com/docs/en/settings
CLI Reference	https://code.claude.com/docs/en/cli-reference
Programmatic Usage	https://code.claude.com/docs/en/headless
Troubleshooting	https://code.claude.com/docs/en/troubleshooting

Agent Skills Documentation

Document	URL
Skills Overview	https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills/overview
Best Practices	https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills/best-practices
Quickstart	https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills/quickstart

Anthropic Example Skills

Available via marketplace: `https://github.com/anthropics/anthropic-agent-skills`

Skill	Purpose
skill-creator	Comprehensive skill authoring guide
mcp-builder	MCP server development guide

Installation:

```
claude /plugin marketplace add https://github.com/anthropics/anthropic-agent-skills
claude /plugin install skill-creator@anthropic-agent-skills
claude /plugin install mcp-builder@anthropic-agent-skills
```

Model Context Protocol

Resource	URL
MCP Introduction	https://modelcontextprotocol.io/introduction
MCP Specification	https://modelcontextprotocol.io/specification/draft
TypeScript SDK	https://github.com/modelcontextprotocol/typescript-sdk
Python SDK	https://github.com/modelcontextprotocol/python-sdk
MCP Servers	https://github.com/modelcontextprotocol/servers

IDE Integrations

IDE	URL
VS Code	https://code.claude.com/docs/en/vs-code
JetBrains	https://code.claude.com/docs/en/jetbrains

CI/CD Integration

Platform	URL
GitHub Actions	https://code.claude.com/docs/en/github-actions

Platform	URL
GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd

Supplemental Resources

Resource	Description
LLM Architecture Framework	Attached document: `llm_architecture_framework.md`
Anthropic Engineering Blog	https://www.anthropic.com/engineering — Technical deep-dives
Trust Center	https://trust.anthropic.com

Appendix C: Knowledge Base Deep Dive [PLACEHOLDER]

> **Status**: To be developed. This appendix will provide comprehensive exploration of advanced knowledge base techniques.

Planned content:

- Knowledge graph integration patterns
- Semantic indexing and metadata tagging strategies
- Atomic content structures for optimal retrieval
- RAG optimization techniques for Claude Code
- Content protocols: examples, micro-templates, rubrics, decision rules, fault trees, agent role cards, lessons-learned
- Scaling strategies for large knowledge bases (100+ files)
- MCP integration with vector databases
- Benchmarking and evaluation methodologies

Appendix D: AI Agents Deep Dive [PLACEHOLDER]

> **Status**: To be developed. This appendix will explore advanced agent architectures and orchestration patterns.

Planned content:

- Agent orchestration patterns
- Multi-agent collaboration frameworks
- Agent state management and persistence
- Hierarchical agent architectures
- Agent evaluation and monitoring
- Integration with official Anthropic skills (skill-creator, mcp-builder)
- Custom hybrid skill development
- Agent lifecycle management

Document Information

Version History:

- v0.5.0-draft (December 2025): Updated all URLs to new domain (code.claude.com), memory system to 5-tier hierarchy with project rules, added built-in subagents, updated MCP scopes, removed duplicate Anthropic skills from plugin
- v0.4.0-draft (December 2025): Updated Agent Skills (6.2) and MCP Integration (6.6) sections based on Anthropic skill-creator and mcp-builder review; added Anthropic Example Skills to references; fixed settings.json naming
- v0.3.0-draft (December 2025): Added privacy/security defaults, KB deep dive placeholder, Agents deep dive placeholder, updated metadata
- v0.2.0-draft (December 2025): Added Git Foundation, Operational Patterns, Placeholder sections; restructured for modularity
- v0.1.0-draft (December 2025): Initial comprehensive guide

Related Documents:

- `case_studies_water_resources.md` — Domain-specific case studies for water resources engineering

Contributing: This guide is intended to evolve. Feedback and contributions welcome via GitHub issues or pull requests.

License: MIT License — Free to use, modify, and distribute with attribution.